



ALGORITHMIQUE et PROGRAMMATION

Introduction au langage Python



UPSSITECH SRI – 1A
Isabelle Ferrané
Isabelle.ferrane@irit.fr

Plan du cours

1- Maquette SRI 2- Pourquoi le choix du langage Python ? 3- Algorithmique et Programmation impérative 3.1- Instructions simples 3.2- Variables et constantes 3.3- Typage dynamique 4- Algorithme : structures de contrôle 4.1- Instruction conditionnelle 4.2- Instruction répétition/itération 5- Algorithme : structures de données 5.1- Structures de données : TABEAU 5.1.1- Tableaux en python via le type <code>list</code> 5.1.2- Tableaux en python via le type <code>array</code>	5.2- Structures de données : ENREGISTREMENT 5.3- Autres structures de données ... 5.3.1- Type structuré natif Python : <code>list</code> 5.3.2- Type structuré natif Python : <code>set</code> 5.3.3- Type structuré natif Python : <code>dict</code> 5.3.4- Types structurés et syntaxe Python 6- Algorithmique : les sous-programmes 6.1- Procédures et Fonctions 6.2- Sous-programmes avec Python 6.3- Passage de paramètres (Algo) 6.4- Passage de paramètres en Python 6.5- Implémentation Python
--	---

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

2

1- Maquette SRI

Semestre 5 (1A)

- **Informatique** Cours commun UPSSITECH (8h)
 - **TD (10h) + TP (16h) Algorithmique - Application avec Python - spécifique SRI**
 - Bases de l'algorithmique
 - Bases des structures de données
- **Programmation impérative**
 - TD (12h) + TP (24h) Algorithmique avancée – Application avec le langage C
 - Structures de données avancées, gestion de la mémoire, pointeurs
 - Compilation séparée
- **PROJET SRI 1** = Projet Fil Rouge Partie 1 (PFR1)
 - Programmation impérative : langage C, commandes et scripts Unix, scripts Python

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

3

1- Maquette SRI

Semestre 6 (1A)

- **Programmation orientée objet** – Cours (12h) Introduction
 - TD (12h) + TP (12h) Appliquée avec le langage Java
 - Concepts associés : classes, méthodes, héritage, ...
 - ...
- **PROJET SRI 2** = Projet Fil Rouge Partie 2 (PFR2)
 - Python, appel à des fonctionnalités développées en Unix/C dans première partie
 - Traitements image / audio
 - Robotique (montage d'une plateforme) et IHM

⇒ en lien avec cours Introduction aux Systèmes Interactifs

Le cahier des charges du projet est sur Moodle : ELURO581 : Projet SRI 1 - Informatique projet

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

4

1- Maquette SRI

Semestre 7 (2A)

- **Programmation avancée** (orientée objet) Appliquée en Python et C++
 - Cours-TD (10h) TP (8h) Python objet
 - Cours-TD (10h) TP (8h) C++ objet
- **PROJET TER**
 - ROS (middleware robotique) et TP Robotique nécessitent Python ou C++

2- Pourquoi le choix du langage Python ?

Python = langage de script orienté objet

Langage Python → syntaxe simple

Script → programme interprété

2- Pourquoi le choix du langage Python ?

Python = langage de script orienté objet

Langage Python → syntaxe simple

Script → programme interprété

Programme = séquence d'instructions réutilisable

mémorisée dans un fichier avec une extension liée au langage (nom_fichier.py)

Interprété = exécuté par un interpréteur (logiciel dédié)

→ pas d'étapes de compilation ni d'édition de liens

Compilation et édition de liens = étapes indispensables avec les langages compilés

C/C++ cf. S5 (avec C) S7 (avec C++)

2- Pourquoi le choix du langage Python ?

Python = langage de script orienté objet

Orienté Objet → type de programmation

... Cf. S6 (avec java) et S7 (avec Python avancé et C++)

Types de programmation = programmation impérative / orientée objet / fonctionnelle / logique
programmation guidée par ... ?

S5 = Focus sur la programmation impérative (ou programmation procédurale)

2- Pourquoi le choix du langage Python ?

Python = langage portable, puissant, interopérable

Portable → s'exécute sur de nombreuses plateformes UNIX / Windows / MacOS / ...
Puissant → simplicité de base + nombreux outils avancés bibliothèques dédiées
Interopérable → liaison avec des composants écrits dans d'autres langages
Intégration Python/C composants C/C++

Prototypage rapide

Mode Développement = concevoir et développer en Python une version simplifiée d'un programme ou d'une application pour prouver la faisabilité (très gourmand en énergie et en temps de calcul par rapport à C)

Mode Production = Remplacer certaines fonctionnalités par des composantes C ou C++ pour améliorer les performances (temps exécution/ mémoire / énergie...)

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

9

3- Algorithmique et Programmation

Algorithmique = méthode et règles générales à appliquer pour formaliser la solution à un problème donné

Etape amont : Analyser et comprendre le problème

Que cherche-t-on à faire globalement ?

Comment se décompose le problème ? Quels sont les sous-problèmes ?

Quelles sont les fonctionnalités identifiées ?

Quels sont les traitements à faire ? Sur quelles données ? Pour quels résultats ?

Quels sont les tests à faire pour valider la solution proposée.

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

10

3- Algorithmique et Programmation

Algorithmme = Description des différentes étapes à mettre en place pour résoudre le problème posé.

Différents niveaux d'analyse = du plus général ... au plus détaillé

→ Raisonnement à un certain niveau d'abstraction de plus en plus fin (raffinement)

SOLUTION ALGORITHMIQUE au PROBLÈME

INDÉPENDANTE du langage de programmation

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

11

3- Algorithmique et Programmation

Programmation = mise en œuvre (implémentation) de la solution (algorithme) dans le langage de programmation choisi (langage cible)

Etape de codage = dépend du type de programmation / de ces spécificités / de la syntaxe du langage choisi

Code source = ensemble de fichiers textes (extension dépend du langage) contenant l'ensemble des instructions à exécuter

Traduction en langage machine (= code binaire) correspondant à des instructions exécutables directement par le processeur

Compilation = production d'un programme exécutable (.exe, .out, C, C++)

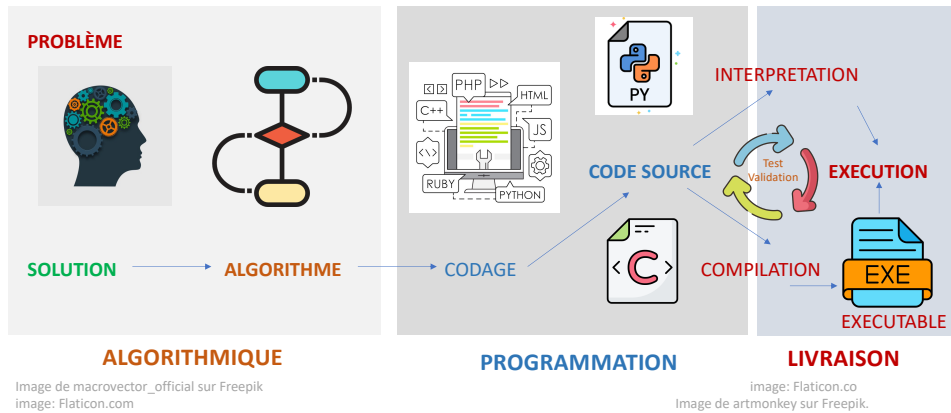
Interprétation = exécution des instructions une à une (via interpréteur, shell, ipython)

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

12

3- Algorithmique et Programmation



3- Algorithmique et Programmation

Focus sur la **programmation impérative**

→ Type de programmation le plus ancien

Programme = séquence d'instructions à exécuter pour réaliser une tâche ou une fonctionnalité donnée ; assimilable à une **recette** (= succession d'étapes)

Séquence = suite ordonnée d'actions à réaliser par la machine

Instruction = action élémentaire (de base dans le langage cible) ou plus complexe (bloc d'instructions, sous-programmes, ...)

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

14

3.1- Algorithmme : instructions simples

Instruction d'entrée / sortie des données

- interaction avec l'**utilisateur** via le terminal (périphériques d'E/S)
 - **Entrée** = saisie d'une donnée au clavier
 - **Sortie** = affichage d'un résultat à l'écran
- traitement des données via un **fichier** stocké en mémoire (disque, clé usb, ...)
 - **Lecture** = récupération d'une donnée dans un fichier (ouvert mode lecture)
 - **Ecriture** = stockage d'une donnée dans un fichier (ouvert mode écriture)

Travailler avec des données = pouvoir mémoriser leurs valeurs (lues ou calculées)

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

15

3.2- Algorithmme : variables et constantes

Notion de variable

Espace mémoire (1 ou plusieurs octets) où est stocké une valeur lue ou calculée lors du traitement

Variable	« boîte » où va être stockée la donnée
Valeur	« contenu » de la boîte
Type	taille de la « boîte » (nb octets) et format du « contenu » (code ascii, virgule flottante,...)
Nom	identifiant symbolique permettant d'utiliser et de retrouver la donnée
Adresse	adresse du premier octet de la zone mémoire contenant la donnée
Déclaration	association d'un nom de variable à un type ex: <code>nb_entier : entier;</code>

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

16

3.2- Algorithme : variables et constantes

Nommage des variables

- caractères non accentués 'a-zA-Z', chiffres '0-9', underscore '_'
- suffisamment long pour être explicite :

l, lg vs **longueur** ou **largeur**

a vs **age** ou **angle**

d vs **distance** ou **duree**

- ne doit pas correspondre à un mot clé (algo/langage)

Bonnes pratiques : vous ne programmez pas que pour vous !!!

Pensez à la **lisibilité** et la **compréhensibilité** de votre code.

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

17

3.2- Algorithme : variables et constantes

Notion de constante

Représente une valeur qui ne doit pas changer au cours du traitement

Constante désigne une valeur non modifiable

Valeur attribuée une fois pour toute lors de la définition de la constante

NOM_CNST en majuscule pour la distinguer des variables ou réutilisation des noms usuels (mathématique/physique) = **bonne pratique**

$\pi \leftarrow 3,1415$ # valeur de la constante π

$c \leftarrow 299792458$ # vitesse de la lumière m/s

Lisibilité du code : distinguer la zone de déclaration des constantes de celle des variables, éviter les déclarations au fil de l'eau

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

18

3.2- Algorithme : variables et constantes

ALGORITHME - TYPES de BASE	Déclarations de variables
entier : domaine des valeurs entières relatives $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$	val1, val2, somme : entier;
réel : domaine des valeurs réelles $\mathbb{R} = \{-\infty, \dots, 0, \dots, +\infty\}$	coord_x, coord_y : réel;
caractère : lettre, chiffre, ponctuation, ... entre quotes : 'a' 'Z' '0' '9' '.' '-' '+' ...	car : caractère;
chaîne de caractères : suite de caractères entre guillemets "valeur_de_ma_chaine"	ch1, ch2 : chaîne de caractères;
booléen : valeur de vérité VRAI/FAUX (TRUE/FALSE)	cond_ok, cond_nok : booléen;

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

19

3.2- Algorithme : variables et constantes

Affectation opération de base qui permet d'attribuer une **valeur** à une **variable**

`nb_entier  $\leftarrow$  15;` # affectation de la valeur 15 à la variable nb_entier

Valeur

peut être donnée directement (expression littérale) ou résulter de l'évaluation d'une expression (calcul, sous-programme)

Expression

fait intervenir des **opérateurs** et un ou plusieurs **opérandes**

Opérateurs

arithmétiques : + - * / modulo

relationnels : = $\neq$ < > $\leq$ $\geq$

logiques : et ou non

Initialisation

première **affectation** d'une **valeur** à une **variable**

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

20

3.2- Algorithme : variables et constantes

Affectation **variable** $\leftarrow$ **expression**; instruction réalisée en deux temps

- 1- **évaluation** de l'**expression** = réalisation du calcul éventuel,
- 2- **mémorisation** de la valeur obtenue dans la **variable** mentionnée c'est-à-dire dans l'espace mémoire dédié

ATTENTION : ERREUR POSSIBLE $\rightarrow$ **incompatibilité des types**

3.2- Algorithme : variables et constantes

ALGORITHME – EXEMPLE (cf. Cours Info)

```
# Déclarations
Constantes
  PI: réel; PI  $\leftarrow$  3,1415 ;
Variables
  rayon : entier;
  perimetre : réel ;

# Instructions = corps du programme
DEBUT
  LIRE (rayon);           # la valeur lue au clavier est affectée à la variable rayon
  perimetre  $\leftarrow$  2 * PI * rayon ; # calcul du perimetre du cercle et mémorisation
  ECRIRE (perimetre);    # la valeur de la variable perimetre est affichée à l'écran
FIN
```

3.2- Algorithme : variables et constantes

ALGORITHME $\rightarrow$ PROGRAMME PYTHON version 0

```
# début
pi = 3.14
r1 = input()
r2 = 2*pi*r1
print(r1,r2)
# fin
```

Quel est le résultat de ce programme python ?

Quels sont les inconvénients de ce programme ?

Comment peut-il être amélioré ?

3.3- Algorithme : typage dynamique (TD1)

Typage dynamique : Quesako ?

Typage

Association d'une **variable** à un **type** de données

- fait de manière **explicite** et une fois pour toute (langage C) = NON modifiable
 - fait de manière **implicite** lors de l'utilisation (python) = **ATTENTION modifiable!!!**
- #### Cast
- Conversion possible de la valeur d'une variable lors de son utilisation
- considère une valeur (variable, expression, résultat d'une fonction, ...)
 - la convertit dans le type indiqué
 - $\rightarrow$ ne modifie pas le type de la variable
 - $\rightarrow$ possible sous réserve de compatibilité de type
 - $\rightarrow$ possible en C, en python, ... Syntaxe différente mais même effet

3.3- Algorithme : typage dynamique (TD1)

Typage dynamique : Quesako ?

possible en python

- fonction de la valeur attribuée à la variable ou à la constante
- lors d'une instruction n'importe quand dans le programme
- la donnée précédente est perdue !!! réaffecte un nouveau type/zone mémoire

Bonnes pratiques : rester rigoureux et éviter tout changement

Voir notebook TD1

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

26

4- Algorithme : structures de contrôle

Structures de contrôle : Contrôler quoi ? Pourquoi ?

Algorithme Description des différentes **étapes** à mettre en place pour résoudre le problème posé.

Etape dépend du niveau de raffinement de l'algorithme
cela va des fonctionnalités très générales (actions générales)
... au niveau d'instruction le plus bas (actions élémentaires) .

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

27

4- Algorithme : structures de contrôle

Exercice

Un robot mobile ne dispose que de 4 actions élémentaires :

`avancer()`, `stopper()`, `tourner()`, `lire_niveau()`

Avancer() se déplacer droit devant d'une distance de 1 mètre et s'arrêter une fois la distance parcourue en conservant la même orientation

Stopper() s'immobiliser et garder la même orientation

Tourner() tourner sur place de 90° vers la droite

lire_niveau() renvoie l'information sur le niveau de batterie (Vrai = OK, Faux = NOK)
`niveau_actuel = lire_niveau();`

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

28

4- Algorithme : structures de contrôle

Exercice

Soit l'algorithme de base permettant au robot de se déplacer en suivant une trajectoire carrée de 1 mètre de côté et de se retrouver à la même position et même orientation qu'au départ.

Initialisation : le robot est orienté dans la première direction à prendre. Le niveau de batterie est OK

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

29

4- Algorithme : structures de contrôle

Algorithme : version 1	
Déclaration niveau_batterie_OK : boolean; niveau_batterie_OK ← Vrai;	Quels sont les inconvénients de cet algorithme ?
Début avancer() ; tourner() ; avancer() ; tourner() ; avancer() ; tourner() ; avancer() ; tourner() ;	
Fin	

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

30

4- Algorithme : structures de contrôle

Que faire si le **niveau de batterie** n'est pas suffisant ?

Il faut pouvoir :

- le connaître → action lire_niveau()
- le conserver → variable niveau_batterie_OK
- le réactualiser → niveau_batterie_OK = lire_niveau();
- le tester → autoriser le déplacement **seulement si** le niveau est **OK**

On a besoin d'une **instruction conditionnelle**

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

32

4.1- Algorithme : instruction conditionnelle

Instruction conditionnelle (simple ou alternative)

SI condition **ALORS** action ;

SI condition **ALORS** action1; **SINON** action2;

Condition expression dont la valeur est de type booléen (Vrai ou Faux)

cela peut être directement un booléen cond_OK

une expression booléenne A < B

ou une combinaison d'expressions booléennes
((A < B) **et** (B < C)) **ou** cond_OK

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

33

4.1- Algorithme : instruction conditionnelle

Exprimer une condition

Opérateurs relationnels

opérateurs qui s'appliquent à des valeurs numériques ou des chaînes de caractères et produisent un **résultat booléen**

= ≠ < > ≤ ≥

Opérateurs logiques

opérateurs qui s'appliquent à des valeurs booléennes pour produire un **résultat booléen**

et ou non

Toute expression conditionnelle est évaluée et la suite dépend de la valeur obtenue
Vrai ou **Faux**

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

34

4.1- Algorithme : instruction conditionnelle

Exercice : conditions et résultats associés

A, B : entier ; A ← 10; B ← 5;
 X, Y : réel; X ← 1.5; Y ← -3.5
 cond_OK, cond_NOK : booléen; cond_OK ← Vrai; cond_NOK ← Vrai;

CONDITION	RES.	CONDITION	RES.	CONDITION	RES.
15 < 20		cond_OK = cond_NOK		A < cond_OK	
3.5 < 0		(A*2) = (B*20)		(non cond_OK) et cond_NOK	
(3*15) > (4*10)		(A-B) = B		(non cond_OK) ou cond_NOK	
'abc' < 'aaaa'		(B + Y) ≤ X		(A < B) et (B < A)	
'A' > 'z'		cond_OK et cond_NOK		('A' < 'B') et ('B' < 'C')	
cond_OK < Vrai		non cond_OK		('B' < 'A') ou cond_OK	

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

35

4.1- Algorithme : instruction conditionnelle

Exercice

Donner une nouvelle version (version 2) de l'algorithme permettant au robot de se déplacer seulement si son niveau de batterie le permet. En cas d'impossibilité un message (significatif) est envoyé à l'utilisateur.

Initialisation : le robot est orienté dans la première direction à prendre.

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

37

4.1- Algorithme : instruction conditionnelle

Instructions conditionnelles – Syntaxe Python

TEST	ALTERNATIVE	CHOIX MULTIPLES
<pre>if condition1 : bloc1 # si la valeur de la condition est Vrai</pre>	<pre>if condition1 : bloc1 # si la valeur de la condition est Vrai else : bloc2 # si la valeur de la condition est Faux</pre>	<pre>if condition1 : bloc1 # si la valeur de la condition1 est Vrai elif condition2 : bloc2 # si la valeur de la condition1 est Faux et que la condition2 est à Vrai elif condition3 : bloc3 # si les valeurs de condition1 et condition2 sont à Faux et condition3 à Vrai ... else : blocN # si aucune des conditions précédentes n'est à Vrai</pre>

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

39

4.1- Algorithme : instruction conditionnelle (TD2)

Instructions conditionnelles

Voir TD2 notebook → TP1

Attention

en python pas de **Début** ni de **Fin** de séquences explicites

→ c'est le niveau d'**indentation** qui définit le bloc d'instructions à exécuter au même niveau (après le :)

```
if test1 and test2 :
    action 1    # à faire si condition vérifiée (True)
    action 2    # à faire si condition vérifiée (True)
    action 3    # à faire systématiquement après le if
```

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

40

4.2- Algorithme : instruction répétition/itération

Que faire si on veut faire **plusieurs tours** ?

Il faut pouvoir :

- **répéter** la même séquence **plusieurs fois**
 - soit un nombre déterminé de fois = connu à l'avance
 - soit un nombre de fois dépendant de la valeur d'une condition (Vrai)
- Faire la même chose en **boucle**
- Sortir de la boucle = **limite fixée atteinte** ou **condition modifiée et non vérifiée**
- **ATTENTION** : une boucle qui ne s'arrête pas est une **boucle infinie = PROBLEME**

4.2- Algorithme : instruction répétition/itération

Instruction itérative Boucle POUR

POUR **N** de VAL1 à VAL2 **FAIRE** action;

N fait office de **compteur** (valeur entière) et prendra les valeurs successives de **VAL1** à **VAL2** éventuellement avec un **pas** (de 1 en 1 par défaut, sinon le pas est spécifié)

POUR **N** de VAL1 à VAL2 **PAS P FAIRE** action;

Bonnes pratiques : ne pas modifier la variable de boucle dans le bloc d'instructions, elle est incrémentée automatiquement de 1 ou du PAS indiqué

4.2- Algorithme : instruction répétition/itération

Exercice

Donner une nouvelle version (version 3) de l'algorithme permettant au robot de se déplacer seulement si son niveau de batterie le permet.

Dans ce cas, il effectuera autant de tours que spécifié par l'utilisateur et signalera la fin du parcours à l'utilisateur.

En cas d'impossibilité un message (significatif) est envoyé à l'utilisateur.

Initialisation : le robot est orienté dans la première direction à prendre.

4.2- Algorithme : instruction répétition/itération

Instruction itérative Boucle TANT QUE

TANT QUE **condition** **FAIRE** action;

Condition doit être **Vrai** pour entrer/rester dans la boucle et faire l'action

- si la condition est fausse à la base, on n'entre pas dans la boucle
- le nombre de passage dans la boucle n'est pas déterminé à l'avance contrairement à la boucle FOR
- la valeur de la condition doit changer au cours des itérations pour provoquer la sortie de la boucle
- si la condition ne change jamais on a une **boucle infinie = PROBLEME**

4.2- Algorithme : instruction répétition/itération

Exercice

Donner une nouvelle version (version 4) de l'algorithme permettant au robot de se déplacer tant que son niveau de batterie le permet.

Dans ce cas, il effectuera autant de tours que possible et il signalera la fin du parcours à l'utilisateur ainsi que le nombre de tours effectués

En cas d'impossibilité un message (significatif) est envoyé à l'utilisateur.

Initialisation : le robot est orienté dans la première direction à prendre.

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

46

4.2- Algorithme : instruction répétition/itération

Instruction itérative Boucle FAIRE

FAIRE action; **TANT QUE** condition

Ici on fait forcément un **premier passage**, pas de condition d'entrée

Condition doit être Vrai **pour rester** dans la boucle et **refaire l'action**

- si la condition est fausse à la base, on fait l'action 1 fois quand même
- le nombre de passage dans la boucle n'est pas déterminé à l'avance contrairement à la boucle FOR
- la valeur de la condition doit changer au cours des itérations pour provoquer la sortie de la boucle
- si la condition ne change jamais on a une **boucle infinie = PROBLEME**

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

50

4.2- Algorithme : instruction répétition/itération

Boucle for (répétition) – Syntaxe Python

Boucle de base	Boucle avec intervalle	CHOIX MULTIPLES
<pre>for k in range(10) : bloc_a_repeter</pre>	<pre>for k in range(3,9): bloc_a_repeter</pre>	<pre>for k in range(3,10,2) : bloc_a_repeter</pre>
# range (10) correspond à un intervalle de 10 valeurs commençant à 0 par défaut → K varie alors de 0 à 9 → 10 non inclus	# range (3,9) correspond à un intervalle de 6 valeurs commençant à 3 → K varie alors de 3 à 8 → 9 non inclus	# range (3,10,2) correspond à un intervalle de 4 valeurs commençant à 3 jusqu'à 9 → K varie de 3 à 9 avec un pas de 2 → 3,5,7,9 → 10 non inclus
# INTERDIT de modifier la variable de boucle dans le bloc	# INTERDIT de modifier la variable de boucle dans le bloc	# INTERDIT de modifier la variable de boucle dans le bloc

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

51

4.2- Algorithme : instruction répétition/itération

Spécification d'un intervalle avec range

range(p, n, q)

génère la suite des entiers de p inclus à n exclu, avec un pas égal à q

- si q > 0 : la suite est construite en ordre **croissant**
- si q < 0 : la suite est construite en ordre **décroissant**

Spécification d'une liste de valeurs

```
for k in liste : # k parcourt tous les éléments de la liste  
    bloc_a_repeter
```

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

52

4.2- Algorithme : instruction répétition/itération (TD2)

Instructions répétitions : boucle for

Voir TD2 notebook

Attention

en python pas de **Début** et de **Fin** de séquences explicites

→ c'est le niveau d'**indentation** qui définit le bloc d'instruction à exécuter au même niveau (après le :)

```
for i in range(10) :  
    action 1      # à faire pour chaque valeur de i  
    action 2      # à faire après action1 même valeur de i  
action 3          # à faire une fois la boucle terminée
```

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

53

4.2- Algorithme : instruction répétition/itération

Boucle while (itération) – Syntaxe Python

```
Boucle de base  
  
# initialisation nécessaire  
while condition :  
    bloc_a_repeter_avec_modification_de_la_condition  
  
# on entre dans la boucle si la condition est évaluée à Vrai  
# on sort quand la condition passe à Faux  
# si elle est à Faux dès le départ, on ne rentre pas dans la boucle  
# si elle ne change jamais on a une boucle infinie  
  
# ATTENTION : les variables qui interviennent dans la condition quelles soient simples  
# ou complexes doivent être initialisées avant et modifiées pendant l'exécution des  
# instructions du bloc
```

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

54

4.2- Algorithme : instruction répétition/itération (TD2)

Instructions répétitions : boucle while

Voir TD2 notebook

Attention

en python pas de **Début** et de **Fin** de séquences explicites

→ c'est le niveau d'**indentation** qui définit le bloc d'instruction à exécuter au même niveau (après le :)

```
while cond1 :  
    action 1      # à faire si condition vérifiée (True)  
    action 2      # à faire après action 2 même condition  
action 3          # à faire après la sortie de la boucle
```

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

55

4.2- Algorithme : instruction répétition/itération

Instructions répétitions : boucle do ... while

Non implémenté directement en python contrairement au langage C

On peut écrire du code qui a le même comportement. Que fait ce code python ?

<pre>code = "163903" nb_tentatives = 0 while True : essai_code = input("Saisie du code ? :") nb_tentatives = nb_tentatives+1 if code == essai_code: print("code bon") break;</pre>	<pre>else : print("erreur") if code != essai_code and nb_tentatives > 3: print("échec") break;</pre>
--	--

08/09/2025

UPSSITECH - SRI - Algorithmique et Programmation

56